

Searching For New Physics with Neutrinos and Machine Learning

Fourth Year MPhys Report

Aryan Sidki

Department of Physics and Astronomy, University of Manchester

(Project completed in collaboration with Hamish McGuire)

(Dated: May 14, 2026)

This project aimed to investigate how differing pretraining strategies of self-supervised deep learning models affected performance on the downstream tasks of deghosting and semantic segmentation on Liquid Argon Time Projection Chamber data. Four different pretraining datasets were explored, comparing self-supervised learning on ghost-free Monte Carlo simulated data, real cosmic data, LArMatch ghost-filtered real cosmic data and the quietest quartile of cosmic events in terms of spacepoint occupancy. Following pretraining, each backbone was adapted using low-rank adaptation fine-tuning for deghosting and semantic segmentation. Model performance was evaluated using the mean Intersection over Union. For segmentation, the strongest performance was shown by the Monte Carlo pretrained model and the LArMatch filtered model, achieving a mean Intersection over Union of 0.821 and 0.808 respectively, indicating that training on cleaner events with less ghost contamination improves adaptability for this task. The quieter events model also showed a relatively strong performance on segmentation, scoring 0.763, demonstrating that lower occupancy events still contain sufficient variation for the model to capture effective representations of point cloud data. In the deghosting task, all models achieved high performance, with a small spread in mean Intersection over Union, suggesting that exposure to all types of ghosts during pretraining is not strictly required for strong performance. However, the model trained on ghost-free Monte Carlo data scored a mean Intersection over Union of 0.884, with the other three models achieving scores above 0.905, suggesting some ghost exposure may be beneficial for deghosting. Overall, the results demonstrate that self-supervised pretraining can learn adaptable representations of Liquid Argon Time Projection Chamber point clouds, while downstream performance is significantly affected by the composition of pretraining data.

1. INTRODUCTION

1.1. Background

Liquid Argon Time Projection Chamber (LArTPC) detectors are commonly used in particle physics experiments for neutrino physics, such as MicroBooNE [1]. These operate by detecting ionisation within a chamber containing a large volume of liquid Argon. When a charged particle such as a product from a neutrino interaction passes through the chamber, it ionises the argon atoms, freeing electrons. The chamber uses a strong electric field to drift these electrons towards three wire anode planes at one end of the detector, orientated in different directions [2]. Each wire plane provides a 2D projection of the particle track, which, in combination with the timing information, is used to reconstruct the 3D positions of ionisation deposits: spacepoints [3].

However, this reconstruction is not perfect, often resulting in the production of ghost points. These occur when the algorithm mismatches signals from different particles, creating fake spacepoints that pollute the reconstructed event, therefore requiring deghosting procedures [4]. Neutrino events often produce thousands of spacepoints. Identifying which of these are ghost points is ambiguous even in smaller volumes; the sheer number of spacepoints in an event makes a practical solution to this task challenging, motivating the development of reconstruction techniques that reduce the number of ghosts [5]. Following ghost removal, for downstream tasks such as particle classification, further difficulties arise due to the scale, sparsity, and complexity of the point cloud data [6].

Hardware based solutions to this problem can be impractical and expensive. For example, increasing the number of wire planes may help improve the reconstruction by reducing the number of ghost points by providing additional views within the detector. However, such approaches increase the cost and complexity of the detector, and larger LArTPC experiments such as DUNE are already substantial and technically demanding systems [7].

An alternative approach is the use of software based solutions. For LArTPC detectors, ghost reduction is tackled by frameworks such as Wire-Cell [8]. These reconstruction frameworks produce the reconstructed 3D ionisation pattern using charge and positional constraints to reduce the number of ghost candidates produced during reconstruction. Following this, a second stage of filtering is used, with the purpose of filtering ghost points that survive the initial reconstruction [9].

On the other hand, particle classification is traditionally performed by studying energy loss through the detector medium, studying data surrounding the Bragg peak [10] [11]. Different particles have characteristic energy loss curves, so this could be used to identify particle type using the Bethe-Bloch equation, which described the average energy loss to ionisation as a function of kinetic energy and mass [12]. Additionally, particles are not confined to the detector's volume, so the Bragg peak and residual range information will often lie outside the detector, making reliable classification of particles significantly more difficult [13].

Although effective, the methods described above have several limitations. For example, ghost filtering within reconstruction pipelines is not perfectly efficient, often losing many genuine points. An example is filtering by LArMatch scores, which range from 0 to 1, where higher scores often correspond to real spacepoints. In the MicroBooNE LArMatch study, applying a threshold score of 0.8 removes points below a score of 0.8, resulting in the removal of 90% of ghost points while retaining 75% of real points [14]. The final particle classification stage is also a difficult task due to scale and ambiguity, for example, when identifying low-energy electrons [15] [16]. This project aims to investigate machine learning approaches that offer potential improvements to current solutions to this problem. The models discussed in this work are pretrained on different datasets representing LArTPC detector events, then undergo further fine-tuning for the tasks of deghosting and particle identification. Performance on these tasks is monitored and compared across models, to assess the effectiveness of different pretraining strategies and how they propagate to performance on downstream tasks. The results offer insight into how changing data composition and ghost exposure during pretraining affects a model’s ability to learn useful, consistent representations of LArTPC events, and how easily these representations can be adapted for related downstream tasks.

1.2. Literature Review

This paper aimed to build on previous work in the field, investigating transformer-based representation learning for sparse 3D data, specifically how pretraining strategies affect downstream task performance on LArTPC data. The wide success of deep learning in spatial recognition tasks such as [17] motivated the extension of its application across a number of fields, such as on unordered 3D point sets [18]. This trajectory was further accelerated by the introduction of the transformer architecture by [19], marking a shift towards attention based representation learning. The potential of transformer based architecture in processing 3D point cloud data was realised in 2021 with Point Transformer [20], which substantially reduced computational expense by restricting attention calculations to local neighbourhoods [21]. Its successors, PTV2 and PTV3, further refined this architecture [22] [23]. This work builds on these developments, investigating a deep learning approach to the downstream tasks of ghost classification and particle class segmentation. Existing MicroBooNE reconstruction workflows use a 2D CNN (convolutional neural network) feature extraction backbone [14]. This work explores an alternative framework, using a self-supervised pretrained 3D transformer [24]. This model directly operates on the 3D candidate point cloud by adapting this model for the downstream tasks of ghost classification and particle class segmentation using LoRA (low-rank adaptation) [25].

2. THEORY

2.1. LArTPC Detector Principles

The LArTPC readout and reconstruction stage described in Section. 1 is summarised at the bottom of Fig. 1. Liquid argon is ionised by charged interaction products, such as neutrino interaction products

or cosmic muons. These ionised electrons drift towards three wire planes due to an electric field. The wire plane signals are combined with timing information to reconstruct the 3D point cloud, containing candidate spacepoints. Reconstruction pipelines then operate on this cloud to remove ghosts or infer information such as particle identity. The models in this work operate at this stage, taking reconstructed point clouds as input, and are fine-tuned to remove ghosts and classify remaining spacepoints by particle class.

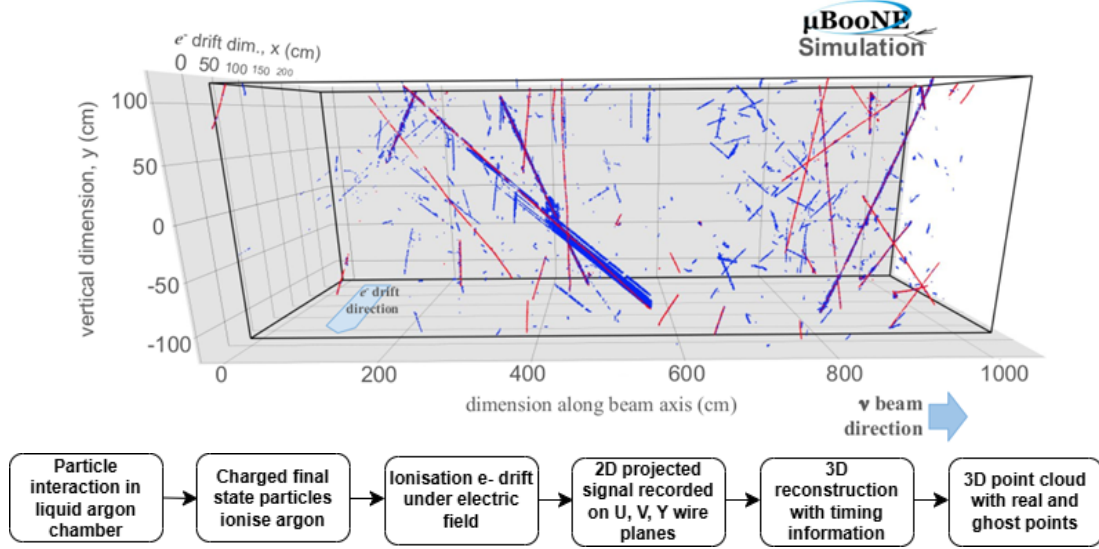


FIG. 1: Top: example simulated MicroBooNE event point cloud with ghost points shown in blue and real points shown in red. Top image adapted from Ref. [14]. Bottom: simplified LArTPC schematic, showing how charged particles ionise liquid argon, producing signals on multiple wire planes which are combined with timing information to reconstruct the 3D point cloud.

This reconstruction produces ghost points: fake spacepoints which do not correspond to real ionisation. The reconstruction uses timing information to match signals measured on one wire plane with signals from another, subject to geometric constraints based on both timing and detector geometry. 2D projections do not uniquely preserve 3D spatial information [26] [27], giving rise to ghost points. Therefore, deciding which signals match is not a unique choice, making filtering to find real spacepoints an inherently ambiguous task. The top of Fig. 1 illustrates this ambiguity, showing a simulated 3D point cloud of an event, where the ghost tracks appear very similar to many of the real tracks. Reconstruction attempts to identify all spacepoints that may be real, to avoid the loss of real data, which then leaves the problem of removing ghosts from the dataset. These are often challenging to identify even in small numbers; in real experiments, there are often thousands of spacepoints, with PILArNet-M, the largest open LArTPC dataset (1M+ events, 5.2B labelled points) containing an average of approximately 5200 points per event [28]. This highlights both the scale and complexity of the problem.

2.2. Events as Point Clouds

The data measured from an event within a LArTPC detector can be represented by a point cloud. The 3D reconstructed set of spacepoints from a given event results in a point cloud of form

$$\{(x_i, y_i, z_i, f_i)\}_{i=1}^N \quad (1)$$

where i is the spacepoint index, N is the number of spacepoints, x, y, z are the three coordinates for spatial positions and f can be any optional feature, such as charge or intensity. N varies for each event, depending on the interaction complexity. Note that an ‘event’ is classed by the detector activity within a given time window, which begins with a trigger signal. As such, an event can contain either one interaction or multiple overlapping interactions [14].

The ordering of spacepoints within the point cloud is arbitrary. Eq. (1) uses the index i to track spacepoints, but the order of them does not matter, this is simply a way of representing the point cloud in a 1D list. Swapping two spacepoints in the list would have no effect, since the information is encoded in the spatial information and correlations between the spatial information of points, along with any additional features [18]. The resulting point clouds will contain both real, physical spacepoints and ghost points.

2.3. Machine Learning Approach

Due to the presence of both ghost and physically valid spacepoints, the point cloud itself contains both valid and spurious structure. The main reason, as discussed, arises from the system producing geometrically consistent but false points. Another is the overlapping of interactions due to the difficulty in actually isolating an ‘event’. This leads to the disambiguation problem: determining which of the candidate points of the cloud are valid.

The transformer is central to this approach to the problem. Elements of the input are mapped to an embedding. This is a vector representation of the element in a high dimensional space. In the context of LLMs, these elements would be tokens (parts of words), whereas point cloud transformers instead operate on points as tokens [29]. Embeddings are useful because through training, the model can learn how to embed such that consistent and similar elements will often have similar vectors. Transformers work by modelling relationships and correlations between elements, updating embeddings accordingly for a context aware representation. This enables a way to determine the consistency of elements with underlying structure. This allows them to be classified, distinguishing physically consistent from spurious elements as a downstream task.

Supervised learning works by training on labelled datasets. A dataset of inputs X has corresponding labels Y . During training, the model learns how to map X to Y by minimising a loss function. This function is a measure of how wrong the predictions $f(X)$ are compared to the true labels Y , acting as a measure of performance. Minimising the loss will lead to a model that approximates the mapping from

X to Y . The limitation here is that labelled events are required. LArTPC data is not labelled in this way, from Eq. (1) each spacepoint only contains position and features which are detected, e.g charge. There is no explicit label associated with the reconstructed spacepoint. For example, it does not say whether it is an electron or muon or even whether it is a ghost or real. Therefore, in order to train this way, Monte Carlo simulations of neutrino interactions are required. Since the event is constructed by the user, every spacepoint has a known origin and hence a label. This is a computationally expensive process, and there is no guarantee it will perfectly replicate detector conditions. Real detector effects are difficult to model perfectly. This motivates the search for other ways to train a model, such as using real detector data.

Self-supervised learning (SSL) works by training on unlabelled datasets, such as reconstructed LArTPC detector data. During training, instead of mapping an input to an output, a subset of the event point cloud is masked, so that the model cannot access it. The model then predicts masked features based on the global structure of the remaining cloud. Again, a loss function is minimised in order to maximise performance. This forces the model to learn representations capturing the global, generalised structure of the data. This makes SSL well suited to LArTPC data, where large volumes of real, unlabelled detector data is readily available.

A problem that arises in 3D point cloud SSL is that models often tend to exploit low level geometric shortcuts during training, such as height and raw coordinate patterns rather than understanding deeper structural representations [24]. This is difficult to avoid, since points within the cloud are distributed sparsely, meaning coordinate information needs to be introduced, enabling the geometric shortcut.

Sonata is a self-supervised model that learns consistent representations of unlabelled point cloud data, such as the LArTPC data. It learns to ‘understand’ the structure of point clouds, by forcing consistency across different views of the same event. To explain this, it is helpful to understand how Sonata was trained.

Sonata uses a teacher-student architecture within its SSL framework. There are two models trained simultaneously: a teacher and a student. Both models are given different views or perturbations of the same event. A different view refers to a variant of the same event, such as with different masking or a different subset of spacepoints removed. With each iteration, a different event is presented to the teacher-student pair, where the teacher and student are presented with slightly different variants. Each model forms an embedding of the event shown. The student is tasked with matching the teacher’s embedding by minimising a loss function that quantifies how well it matches the teacher. Sonata does not simply just compare raw embeddings between the models as a measure of how well they match, instead it uses prototypes. A prototype is a learned cluster in embedding space, so instead of making raw embeddings the same between student and teacher, the model tries to make the student assign a particular point to the same prototype distribution as the teacher. This is actually what is included in the loss function, not just a comparison of embeddings.

The teacher does not learn using loss minimisation. Instead, it follows the student’s updates in weights, but takes much smaller steps per iteration than the student, so its parameters are much more stable throughout training. To be clear, the teacher does not take steps based on the loss, it simply takes a much smaller change in parameters towards the current weights at that iteration of the student. This training is repeated for a massive set of events with different views. When the loss is minimised during training, this enforces a level of consistent agreement between the student and teacher.

In these types of events, physically valid spacepoints are expected to form more stable and coherent structures across different views. On the other hand, ghost points are distributed in a less stable way across perturbations. These are not completely randomly distributed, as they have to be physically plausible to be selected as a candidate spacepoint in the first place, but they appear less consistently and predictably than physically valid points. Although Sonata is not explicitly trained to identify ghosts, the learned consistency of real points may provide a useful prior for the downstream task of ghost removal.

Downstream tasks such as ghost removal require further training known as fine-tuning to the Sonata pretrained backbone described above. Sonata trains the model to learn useful, consistent representations from unlabelled point cloud data. The matrices storing the parameters of large models can contain $O(10^8) - O(10^9)$ parameters. Storing and updating this volume of parameters has large memory and computational requirements [30].

LoRA (low-rank adaptation) is a technique useful for fine-tuning by reducing this cost. Instead of updating the full pretrained parameter matrix W directly, a low rank update matrix ΔW is trained, giving an effective fine-tuned parameter matrix of

$$W' = W + \Delta W \quad (2)$$

where W' is the effective fine-tuned matrix. Rather than learning ΔW as a full matrix, LoRA instead writes it as the product of two smaller matrices that can be trained. During this training W is kept constant, and does not receive gradient updates. $\Delta W \in \mathbb{R}^{d \times k}$ is decomposed into matrices $A \in \mathbb{R}^{r \times k}$ and $B \in \mathbb{R}^{d \times r}$, where $r \ll \min(d, k)$. Here, d and k are the output and input dimensions of the original backbone matrix W , while r is the LoRA rank, controlling the number of parameters that need updating. This reduces the number of parameters that need updating from $N = dk$ to $N = r(d + k)$, and since $r \ll \min(d, k)$, this is a significant reduction in the number of parameters, reducing the cost of training. After training, the full matrix is recovered by multiplying A and B to recover ΔW and this is added to W to obtain the fine-tuned model.

The rank of ΔW is much smaller than that of W . So adding this matrix will only update the action of W within a lower dimensional subspace, as opposed to a full rank update. A key assumption of LoRA is that the useful direction of fine-tuning update lies in a low dimensional subspace. In

principle, the update could actually be high rank. However, the idea is that since fine-tuning is for a specific task, it likely requires a lower rank update matrix to improve performance at a task, since certain directions in embedding space carry a certain meaning after pretraining. This training method has been demonstrated to work across large pretrained models [31] [32].

3. SONATA PRETRAINING

The models investigated in this work were trained on different datasets for pretraining and fine-tuning. The first dataset is real cosmic data from a LArTPC detector. This is unlabelled 3D point cloud data. This data was plotted on a histogram displaying the log-log distribution of total spacepoints per event, shown in Fig. 6, to highlight outliers, since these events were likely reconstruction failure cases, rather than the representative physical events being investigated. The cut-off points for outliers were selected as 3×10^4 and 2×10^6 spacepoints per event. All events outside this range were discarded from the training data. In addition to this, the lowest quartile of events in terms of spacepoints of this outlier-filtered cosmic dataset was isolated to allow pretraining on quieter events.

Ghost-removed datasets were also produced from the outlier-filtered cosmic dataset using LArMatch filtering. In this removal process, LArMatch scores were assigned to the spacepoints in the point cloud of an event. LArMatch assigns a score between 0 and 1 to each reconstructed spacepoint, where a lower score means the point is more likely to be a ghost. These scores are assigned to the data before Sonata sees it during training [14].

For each event passed to Sonata, a threshold score was randomly drawn in the range of 0.15 to 0.75. All spacepoints with a score below the threshold value were removed before passing the point cloud to the Sonata pretraining pipeline. This provided a method of ghost removal, but many ghosts will remain in the data, just as some physically valid spacepoints were likely removed. So each event that Sonata sees will have a different degree of ghost removal.

A separate Monte Carlo simulation of neutrino and cosmic events, without ghost contamination, was also used. The absence of ghosts in the dataset means that this could be used to train a model to see what physically real events look like. However, as mentioned, a key limitation is that this may not perfectly replicate real detector conditions.

In this study, four training methods were investigated, producing four different pretrained models.

Model	Dataset	Ghost exposure	Event selection	Purpose
1	MC neutrino + cosmic	No ghosts	Full	Clean simulated baseline
2	Real cosmic	Ghosts present	Full	Real-data pretraining baseline
3	Real cosmic	Ghosts present	Quieter events	Tests lower-occupancy training
4	Real cosmic v7	LArMatch Ghost removal	Full	Tests ghost-filtered pretraining

TABLE I: Summary of the pretraining datasets and model comparison.

In Table I, the pretraining for each of the models is summarised. Model 1 was trained on the Monte Carlo simulations of neutrino and cosmic events, which did not contain ghosts. This allowed

for the model to be exposed to only clean events and learn the underlying structure of these. Model 2 learned from the entire outlier-filtered cosmic dataset, which contained ghosts. Model 3 was trained on the lowest quartile of the same dataset in terms of spacepoints per event. Model 4 was trained on the v7 LArMatch filtered dataset, with a random ghost removal threshold applied per sample.

The Sonata SSL pretraining stage used a PTV3 (Point Transformer v3) backbone. PTV3 is a transformer-based architecture, designed for efficient point cloud processing. It prioritises simplicity and efficiency, which enables the model to learn relationships between spacepoints while staying scalable for large datasets [23].

The information for every spacepoint of each event is stored in a HDF5 file. This information was the three spatial coordinates and the three pixel values: U, V and Y. These three pixel values essentially represent the amount of charge each of the three wire planes recorded associated with that spacepoint. A highly ionising particle will be described by larger pixel values. This then is passed through a preprocessing pipeline implemented in Pointcept. Here, the data is cleaned and normalised, so that the network is not dominated by just the largest charge deposits. This stage is also where the different views of each event are created, which are fed to the student and teacher when Sonata is pretrained.

The pretraining here was completed over 50 epochs. An epoch means the model has gone through the training dataset once. The base learning rate used was 2×10^{-3} , which controls the size of parameter updates during training. The learning rate decays during training, so parameter updates get smaller as training progresses. The training used a grid size of 0.25 cm, which essentially groups nearby points within this scale, controlling the spatial resolution of point cloud representation. A smaller grid size provides a finer spatial resolution, but this requires more memory. In practice, efficient point cloud architectures are often designed to balance spatial detail with scalability.

3.1. Sonata Pretraining Results

The four Sonata models were pretrained first using the datasets outlined in Table I. Sonata pretraining is self-supervised, meaning there is no way to measure classification performance yet, since this requires labels. Instead, several diagnostic metrics were used to monitor training health. The most important of these metrics is the overall loss, measuring how well the student model matches the teacher targets across different views of events. A decreasing loss indicates that the pretraining objective is being optimised, but will not necessarily guarantee improved downstream performance.

Prototype usage was monitored for both the teacher and student as a pretraining collapse diagnostic, to ensure the model utilised a broad range of prototypes. A prototype is a learned cluster in embedding space, and the model will tend to classify spacepoints into these clusters. If most spacepoints are only assigned to a small number of the available prototypes, this usually suggests representation collapse. The model fails to learn a diverse embedding space, and classifies everything into a small number of categories.

All training was scheduled for 50 epochs. However, since the size of the datasets varied, this required a different amount of time for each model, since an epoch is defined as one run through the full training dataset. As a result of this, due to time constraints, the model trained on ghost-removed data and the model trained on the full cosmic data did not complete the full 50 epochs. As seen in Fig. 2, model 4 reached epoch 18, model 2 reached epoch 26 while models 1 and 3 finished the full 50 epochs. The ghost removal process for model 4 took time, since the full dataset needed to be assigned LArMatch scores. Models 2 and 4 also had to run on a larger dataset than the model trained on quieter events or Monte Carlo data. Fig. 2 shows that all four models successfully optimised the SSL objective, with loss falling rapidly in the first few epochs before slowing later. The models trained on datasets with reduced ghost occupancy achieved lower losses. Lower loss does not necessarily correspond to better downstream performance, since the difficulty of training objective varied depending on the dataset.

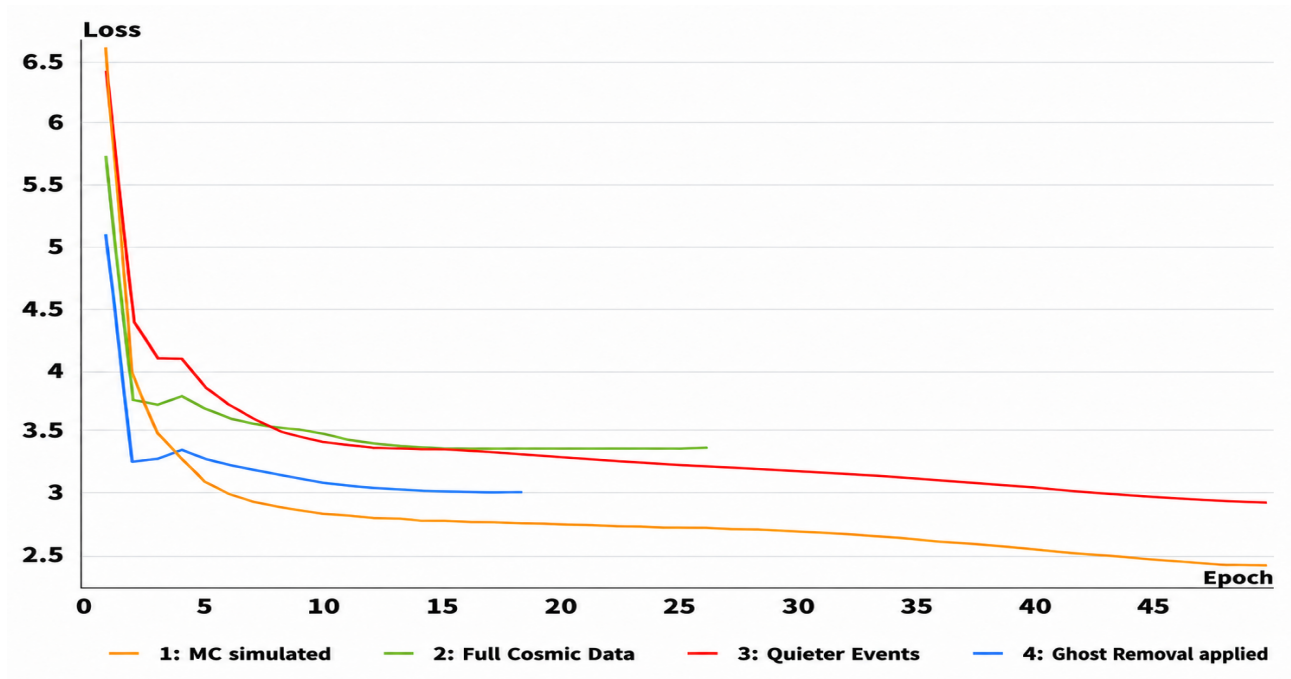


FIG. 2: Pretraining loss curves for the four Sonata models. Produced by the author in Wandb.ai, with modified legend and axis labels for readability.

In Fig. 2, the loss curves for each of the four models is overlaid to allow for an easy comparison. All four models managed most of their optimisation within the first 10 epochs, evidenced by the decrease in loss from 6.6, 5.7, 6.4 and 5.1 for models 1, 2, 3, and 4, respectively. This shows they quickly learned consistent, useful structure on all of the datasets. After this, loss optimisation slowed. Although models 2 and 4 did not complete the full 50 epochs, their loss curves flatten out over the last few epochs, suggesting optimisation may have been done early. This is likely because these datasets contained the highest number of events, so their models processed more data in the fewer epochs.

The LArMatch ghost-filtered model and MC simulated model both maintain the two lowest losses throughout their own training periods, after the initial drop. This suggests that filtering the dataset for ghosts may simplify the self-supervised task by removing inconsistent spacepoints, making it easier

for the teacher-student objective to be optimised quickly. Since the objective is for them to match, removing the less consistent ghost points likely reduces the loss, since a large proportion of the disagreement of teacher-student embeddings likely stems from ghosts, which are typically harder to match with different views.

Due to time constraints, it is difficult to conclude which of the training methods are better, since both model 2 and 4 did not get the full 50 epochs (despite seeming to be optimised), and models 1 and 3 seemed to still have a decreasing loss even at epoch 50. Also, a lower loss in pretraining may be partially due to differences in the training distribution.

Model 1 quickly optimised the Sonata objective with its dataset, likely because of the fact that this data is ghost-free, meaning it is easier for the model to identify useful structure with noisier data points removed. This model seemed to have the lowest floor for loss of the models, reflecting its training dataset, which likely made it easier to learn consistent embeddings, since the events used are clean simulations.

Model 2 exhibited a stable loss after around 13 epochs, staying within the bounds of 3.306 and 3.322. This is likely because with the full cosmic dataset (including ghosts), the model processes a larger set of the general features of LArTPC point clouds, allowing it to quickly learn useful representations early on, utilising a broader spread of the prototypes. It would be interesting to see how this loss would evolve over the full training process, since that would show whether it had learned the majority of useful features of these point clouds early on, or if there are deeper features it can learn throughout a longer training process. The fact that it plateaus at a higher loss than the other models is likely because of the fact that this dataset is more complex, containing both extra noise from ghosts and higher frequency events. Interestingly, the counts for the number of used prototypes across all models, shown in Figs. 7 and 8, were similar for the number of events processed, suggesting all four datasets allow Sonata to learn a broad prototype space.

The loss for model 3, trained on quieter events, decreased continuously until epoch 50, reaching the lowest loss of the three models trained on real data. This loss takes around 40 epochs to get as low as the LArMatch filtered model 4, suggesting that this quieter dataset contains less structural diversity, resulting in the model needing to process the dataset more times to achieve the same level of consistency. This lack of variation in the data likely provides a more stable training regime, where loss is optimised slowly over a longer period. Interestingly, this continued fall in loss also shows that quieter events are not ‘too simple’ to learn meaningful representations, as the model was still learning at the end of the 50 epochs. It is worth noting that this model has had to learn consistent representations in the presence of ghost points, a somewhat more complex task, which would contribute towards extended optimisation time.

4. LORA FINE-TUNING AND DOWNSTREAM PERFORMANCE

The four Sonata pretrained models then separately underwent supervised LoRA fine-tuning for specific downstream tasks. There were two main tasks they were trained on: ghost removal and 8-class semantic segmentation. Both of these were trained using a Monte Carlo simulated dataset containing data labelled with the particle class and a truth label on whether the spacepoint is a ghost or not. This dataset was shuffled and divided into a training set and a validation set, which was used to test the performance of the model. The four pretrained backbones produce embeddings that already encode useful information in terms of data structure. However, the purpose of the fine-tuning is to make this embedding quality more aligned towards classification within the specific task. All four backbones were fine-tuned in the same way, giving a way to compare how the different methods of pretraining affects learning of these specific downstream tasks. Note that the pretrained backbone matrices used here contain approximately 10^8 parameters, motivating the use of parameter-efficient LoRA fine-tuning over full-parameter fine-tuning.

Both fine-tuned models were tested on unseen events from the validation dataset, and with accuracy on predicting validation events plotted as a function of training epoch. Both the validation accuracy and loss was monitored throughout training, to assess progress, but also to detect overfitting. Overfitting is where the model learns specific features of the training set rather than general classification features, which is the reason for testing against the validation set. Validation is important at this step, since supervised learning has been shown to be more prone to overfitting than SSL, motivating monitoring [33].

In the fine-tuning for deghosting, the point cloud of an event was passed into the model, and the backbone produced embeddings for the spacepoints. A task-specific head then predicted a label for each point based on the embedding. A head is the final prediction layer after the pretrained backbone. The backbone produces an embedding, and the head maps each embedding to a set of class scores, which determine which class the model will predict this spacepoint in. For ghost removal, this is a 2-class head, for real and ghost. For segmentation, it is an 8-class head, for each of the eight particle classes. After this, the predicted labels were compared with the ghost truth labels in the dataset, and a supervised loss was calculated. The loss is used to update the LoRA update matrices, while the backbone parameters remain frozen.

For segmentation, fine-tuning used the same MC dataset used for deghosting, except with ghosts removed. They were filtered out using their truth labels before training, providing isolation of the classification task from ghosts. The model was trained to predict the particle class of each spacepoint, from one of eight semantic labels: electron, muon, pion, proton, gamma, Michel electron, delta ray or low-energy deposition. These labels correspond to the SSNet style label scheme used in the dataset [6]. Here, Michel electrons refer to electrons produced by muons decaying at rest, while delta rays are

energetic secondary electrons produced by ionisation, appearing with small branches along a charged particle’s track [34] [35]. Low-energy deposition (led) is a broad class for smaller energy deposits that are not treated as distinct particle instances [36]. Just as for deghosting the training attempted to minimise the loss when predicting the particle labels of spacepoints within events.

This fine-tuning was done over 100 scheduled epochs, since LoRA uses fewer trainable parameters for a more specific task, so was given more epochs to optimise over. The grid size used was again 0.25 cm, and the base learning rate was 5×10^{-4} . The backbone is already pretrained in making useful embeddings, so a lower base learning rate was used to avoid destroying these learned representations. The LoRA parameter α and rank r were chosen to be 32 and 16, respectively. The rank r controls the size of the low rank update. A higher rank gives more flexibility, but also more parameters to train. The LoRA α is a scaling factor that controls the size of the LoRA update.

Following pretraining, all four backbones were fine-tuned in the particle segmentation and deghosting task for a scheduled 100 epochs. LoRA fine-tuning used labelled Monte Carlo data, allowing a direct evaluation of task performance. The model was periodically evaluated on the validation dataset while being trained on the training dataset. Validation accuracy was used to monitor the evolution of performance on unseen events on both tasks. A limitation of this metric is that it can be misleading for classification tasks, as it rewards bias towards the dominant classes.

In order to mitigate the limitations of using the validation metrics alone, a useful metric to monitor is IoU (intersection over union). It is calculated by

$$\text{IoU} = \frac{\text{TP}}{\text{TP} + \text{FP} + \text{FN}}, \quad (3)$$

where TP, FP and FN stand for true positive, false positive and false negative predictions for a given class. IoU measures the overlap between the set of points predicted to belong to a class and the set of true points in that class. The denominator includes both false positives and false negatives, penalising misclassifications.

The mIoU is calculated as the mean of the IoU for each category the model is classifying spacepoints into. Due to time constraints, not all fine-tuning runs completed the full scheduled 100 epochs. As a result, models 3 and 4 may not have fully converged, meaning that downstream performance could potentially be improved with a longer training time. All models, except model 1, suffered from incomplete pretraining or fine-tuning, or both. This adds a caveat to any discussion points below, as the varying training duration likely contributed to the results. Mean IoU plots are provided as a function of epoch in the following subsections for the models pretrained on real data. The model pretrained on the Monte Carlo dataset is not included since this data was not available for its fine-tuning, as it was run on a separate Wandb workspace. The fine-tuning results are displayed in Table II. For the segmentation task, model 1 performed the best at a mIoU of 0.821, followed closely by model 4, model 3, then model 2, which performed noticeably worse, with a mIoU of 0.654. The deghosting

Model	PT Epochs	Segmentation		Deghosting	
		FT Epochs	mIoU	FT Epochs	mIoU
1: MC Simulated	50	100	0.821	100	0.884
2: Full Cosmic	26	100	0.654	100	0.916
3: Quieter Events	50	42	0.763	54	0.906
4: LArMatch Filtered	18	43	0.808	54	0.919

TABLE II: Summary of the pretraining and fine-tuning results of mIoU for each model. PT denotes pretraining and FT denotes fine-tuning.

models saw a much smaller spread in mIoU, with the lowest at 0.884 for model 1, and the highest at 0.919 for model 4. The mIoU was greater than 0.654 and 0.884 for segmentation and deghosting respectively. This demonstrates that the Sonata architecture works well for this set of LArTPC tasks, regardless of pretraining method. These results show that models 1 and 4 were the most adaptable overall to both tasks, suggesting that pretraining with fewer ghosts may be optimal for training a versatile backbone that can be adapted for both tasks.

In the following discussion, the validation mIoU is used as the primary performance metric, since it is more robust against the model biasing dominant classes. This is the mIoU calculated when measuring model performance on unseen events from the validation dataset. Additionally, a t-SNE plot is provided for one of the particle segmentation models to illustrate how the fine-tuned model represents point cloud events by forming clusters of spacepoints with particles of same class. The deghosting model discussion does not contain such a plot, as for binary classification, the quantitative metrics discussed such as mIoU are more useful for comparing models.

4.1. Particle Semantic Segmentation Interpretation and Discussion

Among the four models, model 1 (MC simulated) and model 4 (ghost filtered) achieved the best segmentation performance. It is worth noting that model 1 was the only model pretrained on MC data which may differ from real LArTPC data in subtle but potentially important ways. Since the fine-tuning was also done on a MC dataset, this backbone may be more suited to performance on simulated data, which may not necessarily be carried forward on real events. Another reason is that it was the only model that completed the full scheduled training period for pretraining and fine-tuning. These factors likely explain the higher performance, so the results should not necessarily be interpreted as model 1 having a superior pretraining method. A caveat to this is that the MC dataset it was pretrained on was of a considerably smaller volume than the real data used for the other models, meaning that, while it completed 50 full epochs, it saw a much smaller sample size of events.

Model 4 had the fewest overall training epochs across pretraining and fine-tuning, yet demonstrated a performance comparable to that of model 1, while clearly better than models 2 and 3. If given an extended training period, model 4 could potentially perform the best of the four. Its performance may

be attributed to pretraining on data with a reduced ghost population, which is discussed below. It is difficult to conclude the reason behind the relatively poor performance of model 2, since its pretraining was incomplete. The reason could simply be that it started fine-tuning with the weakest backbone, since it had the highest pretraining loss. This could be due to unfinished training or because the model struggled more in making consistent representations when shown the full cosmic dataset. The latter suggests that training on a broader spectrum of ghost-populated, higher occupancy events, may result in lower downstream performance in segmentation.

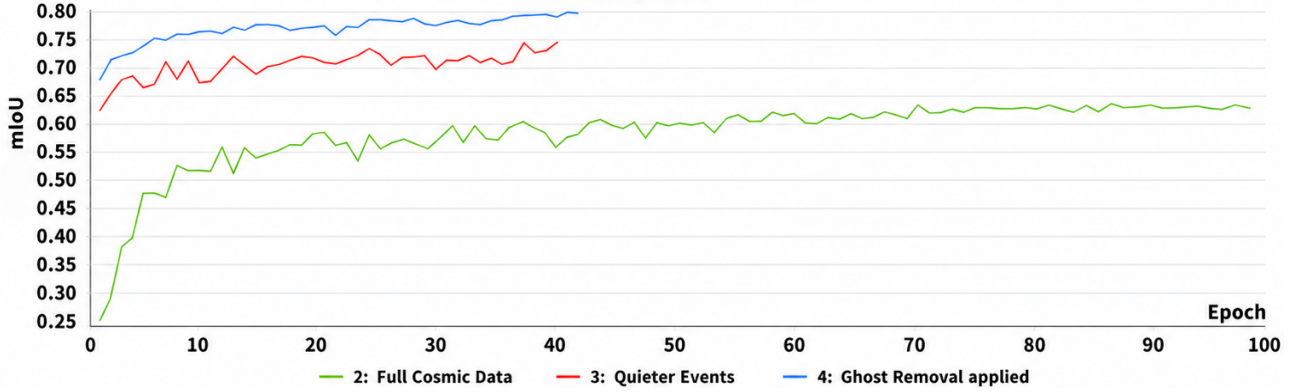


FIG. 3: Validation mIoU as a function of fine-tuning epoch for semantic segmentation. The curves compare the available real data pretrained models during LoRA fine-tuning. Produced by the author in Wandb.ai, with modified legend and axis labels for readability.

The curves in Fig. 3 show mIoU as a function of epoch for the models trained on real data. Models 2 and 4 (ghost filtered) appear to be settling in terms of mIoU, with model 4 still slightly increasing. Model 3 is still showing an upward trend, suggesting it still had room for improvement, provided with more training time. However, further training could still result in improvement for model 4, since it has only settled for around five epochs. Model 4 also showed a similar behaviour in pretraining, where its loss was close to optimisation earlier than the other models, at around epoch 18. This supports model 4 as a strong candidate for this task, since it requires the least overall training for strong overall results. However, this requires LArMatch ghost filtering to be applied prior to pretraining.

If the results for all four models are taken at face value, the results indicate that training on data with fewer ghosts corresponds to better segmentation performance. The models trained on full ghost occupancy performed worse, with model 2 being even worse than model 3. This is likely because it seemed to have the weakest backbone, based both on its loss during pretraining and on its performance here. Model 4’s mIoU also seems to settle at an earlier epoch, suggesting that pretraining on ghost filtered data may result in earlier convergence for segmentation.

The results of Table II also suggest that the models do not need to pretrain on neutrino events to perform downstream segmentation tasks. The real cosmic dataset used by models 2, 3 and 4 was not explicitly composed of neutrino interaction products. This shows that these models learned useful, generalised representations of detector physics, allowing adaptation towards segmentation. However, stronger alignment between pretraining and the task is still beneficial, as evidenced by models 1 and

4 performing better after pretraining on fewer ghosts.

These results do not necessarily imply models 1 and 4 would perform the best at segmentation on a class filled with ghosts, since that is not tested here. Also, in Fig. 3, the trajectory of model 3’s mIoU indicates a possibility of it overtaking even models 1 and 4. This would contrast the above argument if it were to overtake them, as it would suggest that training on quieter events may be a bigger factor than lower ghost occupancy for segmentation. However, this discussion point would require further training time to confirm. Nonetheless, even if not fully optimised, model 3 still performed well on segmentation, demonstrating that pretraining on quieter events is effective for downstream performance. In fact, its performance being significantly higher than model 2’s indicates that for pretraining with full ghost exposure, training on quieter events is actually beneficial for segmentation.

In addition to the performance metrics, t-SNE (t-distributed Stochastic Neighbour Embedding) was used as a visualisation tool to represent the high-dimensional embedding space as a 2D projection. This facilitates a way to see the clustering of embeddings produced by the model. This method works by calculating the Euclidean distance between the embeddings of two spacepoints, converting this into a probability, which represents how likely they are to be neighbours. Next, t-SNE produces a 2D projection of the spacepoints. In this projection, t-SNE tries to match the neighbourhood probabilities to the probabilities in embedding space. This preserves the structure and local relationships between spacepoints [37]. This has been applied in embedding global point cloud features into a 2D space to visualise the correlation between point clouds [38]. The projected points are coloured by particle class, so a strong performing model should form separated clusters of spacepoints for each colour.

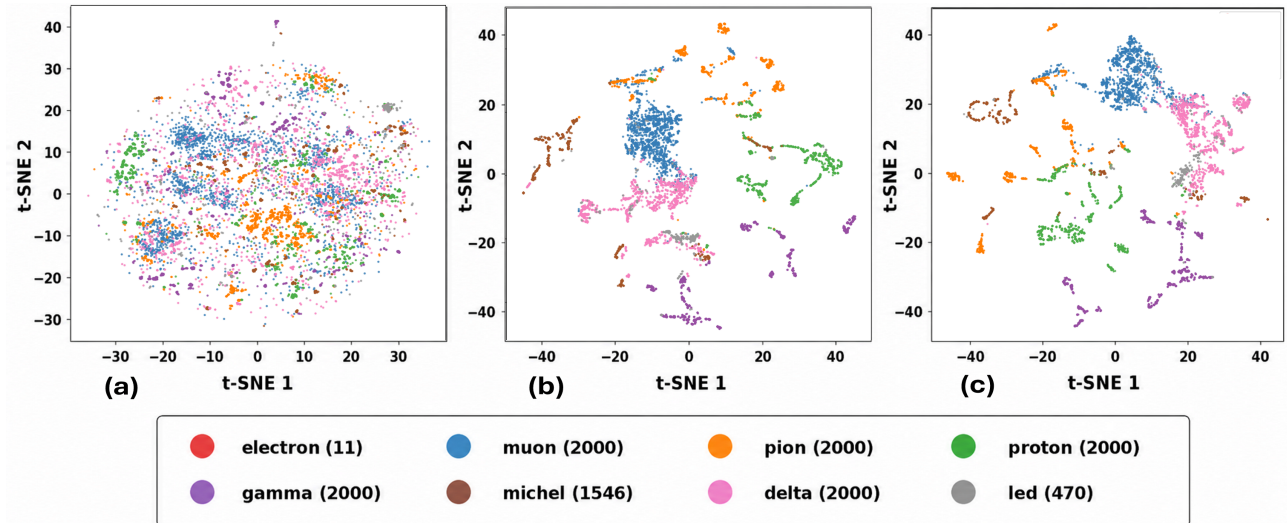


FIG. 4: t-SNE projections of model 4’s Sonata backbone features at three stages of LoRA fine-tuning for semantic segmentation: (a) pretrained backbone, (b) epoch 2, (c) epoch 43. Produced by the author in Python.

In Fig. 4, the t-SNE visualisations for model 4 (LArMatch filtered) are shown before, after 2 epochs and after 43 epochs of LoRA segmentation fine-tuning, with points coloured by particle class. The backbone clearly holds some useful representation, showing some clusters of the same particle type,

even if not the most well defined. Interestingly, by only epoch 2, the model already shows strong signs of clustering, with clear, distinct clusters of certain particle types shown. Comparing with Fig. 4(c), it is difficult to tell by inspection which would be the best performing model for segmentation. These observations are consistent with the curves in Fig. 3, which are shown from epoch 1, showing that for model 4, the mIoU is already as high as 0.7, only reaching just above 0.8 after full fine-tuning. This strong performance in epoch 1 is not because the backbone is good at segmentation, but because the backbone is easily adaptable to this task, with enough learning over 1 epoch to perform well. This plot was produced using t-SNE for segmentation with a perplexity hyperparameter of 100, which controls the balance of preserving local and global structure in the plot. A higher perplexity biases the embedding towards global structure. The absolute positions of clusters in t-SNE space is essentially arbitrary, which likely explains the movement of clusters from the epoch 2 plot to epoch 43, rather than a large change in parameters.

4.2. Deghosting Interpretation and Discussion

The deghosting performance demonstrated a much lower spread in terms of mIoU, as seen in Table II and Fig. 5. This likely stems from multiple reasons. One being that the binary classification task is naturally simpler compared with the segmentation task. The IoU penalises misclassifications and mIoU is found by the mean over classes, so a smaller number of classes decreases the number of ways misclassification could occur.

Another reason why the spread was smaller was the pretraining configurations. Models 1 and 4 were trained with ghost removed or filtered datasets. For models 2 and 3, they were trained on data with full ghost presence. For these models, during pretraining, an optimised model would have a far greater degree of agreement between teacher and student for predicting real points than ghosts, due to their consistency. As a result, all four backbones likely have far more consistent representations of real points than ghosts, so they serve as a strong starting base for deghosting. Fig. 5, supports this, since for the available models 2, 3 and 4, we see a score close to 0.9 for mIoU from epoch 1, much higher than the initial segmentation scores. Of course, this is partly due to complexity of the task, but also because the pretraining was likely more closely aligned with the deghosting task, meaning it required fewer training epochs for similar results. The small spread in mIoU indicates that the models do not need to have processed ghost points to be adaptable for deghosting. However, some ghost exposure is likely beneficial, since the only model that was not exposed to any ghosts was model 1, which had the lowest mIoU.

Additionally, the results indicate that the models do not need to have processed all types of ghosts either. As mentioned, model 4 was trained on data with ghosts randomly removed from each event with a randomly sampled LArMatch threshold between 0.15 to 0.75. This model did not see ghost points with a score below 0.15 prior to fine-tuning, since all runs remove points with scores under this, yet showcased the best performance on deghosting. Also, if some types of ghosts are only present

in busier events, model 3 may not have seen all ghost types, yet still showed strong performance. Model 3’s strong performance also shows that pretraining on quieter events is effective for downstream deghosting. Prior to testing, it was unsure whether quieter events would contain the diversity required for model 3 to perform when exposed to the types of events it had not seen before. However, models 2 and 4 performed marginally better than model 3, suggesting that there could be some benefit from exposure to all types of events.

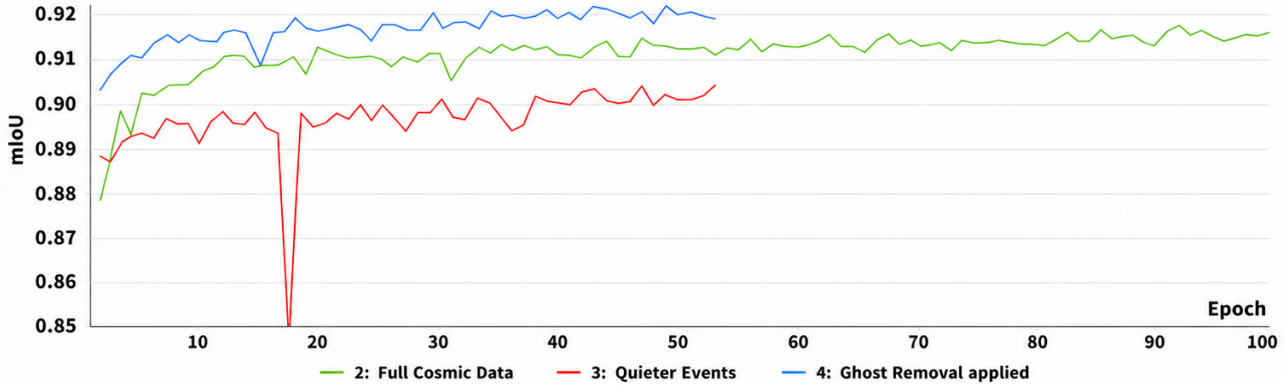


FIG. 5: Validation mIoU as a function of fine-tuning epoch for deghosting. The curves compare the available real data pretrained models during LoRA fine-tuning. Produced by the author in Wandb.ai, with modified legend and axis labels for readability.

Inspecting Fig. 5, at epoch 18 there was a large drop in mIoU for model 3, likely due to poor parameter updates, resulting in the over prediction of either class for a period of this epoch, resulting in an increased mIoU denominator. At that stage of training, the learning rate was still quite high, which can result in a bad parameter update if presented with a batch of events that differs a lot from the average. The mIoU had completely recovered by the following epoch, which likely means that by the end of epoch 18, the mIoU must have mostly recovered, not affecting the following epoch.

Comparing models, model 4 achieved the highest deghosting performance, followed by models 2, 3 and 1. Despite a strong result, model 1 performed the worst of the four. The most reasonable explanation for this is that this model was never exposed to ghosts in pretraining. For the same reason model 1 performed the best on segmentation, it is the weakest in deghosting, since the pretraining dataset did not contain ghosts. It still scores highly, likely because it is being fine-tuned with MC data again, so it should be able to consistently represent real points at least, which is a strong starting point. Model 2 performed significantly better on deghosting than segmentation. This is likely because during pretraining, model 2 was exposed to all events, even the busiest, with ghosts. The two worst performing models were constricted in terms of exposure to events in pretraining, with model 1 not shown ghosts, and model 3 only seeing quieter events. The small discrepancy between these models and models 2 and 4 is probably mainly due to this. Models 2 and 4 had both seen the busiest events, giving these models the edge over model 3. Similarly, model 1 had never encountered ghosts before,

giving the other three models an advantage over it. Model 4 ended with a higher mIoU than model 2 after only 54 epochs compared to model 2's 100 epochs. This alone would suggest that model 4's pretraining was superior, however, inspecting Fig. 5, it can be seen that model 4 had settled for over 10 epochs. This suggests that model 4 probably would not benefit from extended training, while models 2 and 3 seem to still be climbing in mIoU. With an IoU of around 0.92, model 4 seems to be the superior one in that it performs as high as model 2 within half the epochs, meaning again the backbone seems well aligned with the given task.

An interesting comparison is model 1 and 4, which are the worst and best despite both being pretrained on data with lower ghost occupancy. Model 4's main advantage over model 1 is that its backbone has processed significantly more events than model 1, allowing it to capture many variations of similar events in its representations. This means that when fine-tuned, it will likely perform better on the validation set as it has processed a broader range of events. This is further supported by the improved performance of model 2, since it was exposed to the same dataset but with full ghost occupancy. Another advantage of model 4 is that the backbone had processed some ghosts. The amount of ghosts removed by LArMatch filtering is random with each event. Often there will be many ghosts remaining, so when model 4 was pretrained, it likely had some learned representations of what ghost points look like. So, early on in the supervised fine-tuning, when it will process labelled data, it will quickly associate these prototypes with ghosts. The fact these ghosts remain after filtering also indicates these are the more persistent, ambiguous ghosts. So, if the model indeed managed to form prototypes for these types of ghosts, it will immediately perform better than the other models, since these ghosts are probably the easiest to misclassify.

An interesting discussion point is the performance comparison of these models to the current LArMatch score-based ghost filtering. As mentioned in the introduction, the LArMatch method for a ghost threshold of 0.8 removes around 90% of ghosts while retaining 75% of real points. The fine-tuned models discussed here had mIoUs ranging from 0.884 to 0.919, suggesting that transformer-based representations can rival established filtering methods in reconstruction pipelines. A more precise comparison would require evaluation on the same labelled dataset with a known ghost to real point ratio under identical conditions, which is left for future work.

5. CONCLUSION

This work investigated how different pretraining strategies affect downstream LArTPC point cloud processing tasks using the Sonata framework. Across both tasks, the strongest performing models were models 1 and 4, which may indicate that pretraining on fewer ghosts results in better adaptability for downstream tasks. The real-cosmic models suggest that explicit neutrino events are not strictly required during self-supervised pretraining to later adapt and achieve effective segmentation performance on neutrino events. Additionally, for segmentation, training on events with fewer spacepoints

may provide a benefit for this task, supported by the fact model 3 outperformed model 2. Although all four models adapted well to deghosting, model 1 performing the weakest suggests that some degree of ghost exposure during pretraining may still be beneficial. Models 2 and 4 performed the best, demonstrating that adaptability to deghosting was not affected if all types of ghosts were not seen during training. Model 3 performed well on both tasks, despite likely not being fully optimised due to time constraints. This showed that training on quieter events was sufficient to capture the general features of LArTPC point clouds. However, the fact that model 3 was likely still not fully optimised after 50 epochs of pretraining indicates that these events may suffer from a lower degree of diversity, hence requiring more training epochs to fully optimise and learn features. Overall, these results demonstrate that the Sonata framework is well suited to learning adaptable representations for these downstream LArTPC tasks. In particular, the final performance of the models in deghosting in terms of mIoU suggests that this transformer based architecture can rival current ghost filtering methods such as the LArMatch threshold. A natural extension to this work would be to repeat the studies with complete training runs. Future work could compare the epiplexity [39] of pretraining datasets to quantify their learnable structural content. The learned representations could also be extended beyond deghosting and segmentation towards higher level neutrino event selection, such as anomaly detection for rarer events.

6. REFERENCES

-
- [1] R. Acciarri *et al.*, Journal of Instrumentation **12**, P02017, arXiv:1612.05824.
 - [2] J. Joshi and X. Qian, arXiv preprint arXiv:1511.00317 (2015), arXiv:1511.00317 [physics.ins-det].
 - [3] H. Yu *et al.*, Journal of Instrumentation **15** (10), P10017, arXiv:2007.12743.
 - [4] L. Domine, *Deep Learning Based LArTPC Event Reconstruction and Nonmagnetic Muon Sign Determination in the ICARUS Detector*, Ph.D. thesis, Stanford University (2023).
 - [5] X. Qian *et al.*, Journal of Instrumentation **13** (05), P05032, arXiv:1803.04850.
 - [6] P. Abratenko *et al.*, Physical Review D **103**, 052012 (2021), arXiv:2012.08513.
 - [7] B. Abi *et al.*, Journal of Instrumentation **15**, T08008, arXiv:2002.03005.
 - [8] P. Abratenko *et al.*, Journal of Instrumentation **17**, P01037, arXiv:2110.13961.
 - [9] P. Abratenko *et al.*, Journal of Instrumentation **16**, P06043, arXiv:2011.01375.
 - [10] W. H. Bragg, Philosophical Magazine 6, **8**, 719 (1904).
 - [11] W. H. Bragg and R. Kleeman, Philosophical Magazine 6, **10**, 318 (1905).
 - [12] H. Bethe, Annalen der Physik **397**, 325 (1930).
 - [13] P. Abratenko *et al.*, Journal of High Energy Physics **2021**, 153 (2021), arXiv:2109.02460.
 - [14] MicroBooNE Collaboration, *Reconstruction and Selection of Neutrino Interactions in MicroBooNE using Deep Convolutional Neural Networks*, Tech. Rep. MICROBOONE-NOTE-1123-PUB (Fermi National Accelerator Laboratory, 2021).

- [15] DUNE Collaboration, *Physical Review D* **107**, 092012 (2023), arXiv:2210.09905 [hep-ex].
- [16] MicroBooNE Collaboration, *Journal of Instrumentation* **12** (09), P09014, arXiv:1704.02927 [physics.ins-det].
- [17] A. Krizhevsky, I. Sutskever, and G. Hinton, *NeurIPS* (2012).
- [18] C. R. Qi *et al.*, *CVPR* (2017).
- [19] A. Vaswani *et al.*, *Advances in Neural Information Processing Systems* (2017).
- [20] H. Zhao *et al.*, *ICCV* (2021).
- [21] P.-S. Wang, arXiv preprint arXiv:2305.03045 (2023), arXiv:2305.03045 [cs.CV].
- [22] X. Wu, Y. Lao, L. Jiang, X. Liu, and H. Zhao, arXiv preprint arXiv:2210.05666 (2022), arXiv:2210.05666 [cs.CV].
- [23] X. Wu, L. Jiang, P.-S. Wang, Z. Liu, X. Liu, Y. Qiao, W. Ouyang, T. He, and H. Zhao, in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2024) arXiv:2312.10035 [cs.CV].
- [24] X. Wu, D. DeTone, D. Frost, T. Shen, C. Xie, N. Yang, J. Engel, R. Newcombe, H. Zhao, and J. Straub, arXiv preprint arXiv:2503.16429 (2025), arXiv:2503.16429 [cs.CV].
- [25] E. J. Hu *et al.*, arXiv preprint arXiv:2106.09685 (2021).
- [26] R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd ed. (Cambridge University Press, 2003).
- [27] H. S. Park, T. Shiratori, I. Matthews, and Y. Sheikh, in *Computer Vision – ECCV 2010*, Lecture Notes in Computer Science, Vol. 6313 (Springer, 2010) pp. 158–171.
- [28] S. Young, Y.-J. Jwa, and K. Terao, arXiv preprint arXiv:2502.02558 (2025), arXiv:2502.02558 [hep-ex].
- [29] X. Yu, L. Tang, Y. Rao, T. Huang, J. Zhou, and J. Lu, in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2022) pp. 19313–19322, arXiv:2111.14819 [cs.CV].
- [30] N. Houlsby, A. Giurghi, S. Jastrzebski, B. Morrone, Q. de Laroussilhe, A. Gesmundo, M. Attariyan, and S. Gelly, in *Proceedings of the 36th International Conference on Machine Learning*, Proceedings of Machine Learning Research, Vol. 97 (PMLR, 2019) pp. 2790–2799.
- [31] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, *Advances in Neural Information Processing Systems* **36**, 10088 (2023), arXiv:2305.14314 [cs.LG].
- [32] S. Wang, X. Liu, L. Kong, J. Xu, C. Hu, G. Fang, W. Li, J. Zhu, and X. Wang, in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2025) pp. 6605–6615, arXiv:2504.16023 [cs.CV].
- [33] L. Ericsson, H. Gouk, and T. M. Hospedales, in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2021) pp. 5414–5423, arXiv:2011.13377.
- [34] L. Michel, *Proceedings of the Physical Society. Section A* **63**, 514 (1950).
- [35] S. B. Curtis, *NASA Three-Dimensional Tissue Models for Space Radiation Risk Assessment* (2018).
- [36] S. Young and K. Terao, arXiv preprint arXiv:2512.01324 (2025), arXiv:2512.01324 [hep-ex].
- [37] L. van der Maaten and G. Hinton, *Journal of Machine Learning Research* **9**, 2579 (2008).
- [38] B. Zhang, W. Huang, S. Raghupathi, I. Chandratreya, and Q. Zhang, in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops* (2019).
- [39] M. Finzi, S. Qiu, Y. Jiang, P. Izmailov, J. Z. Kolter, and A. G. Wilson, arXiv preprint arXiv:2601.03220 (2026), arXiv:2601.03220 [cs.LG].

A. SUPPLEMENTARY PLOTS

This appendix contains supplementary plots used to support dataset selection and monitor model training behaviour. These plots are provided for completeness and were not used as the primary basis for downstream model comparison.

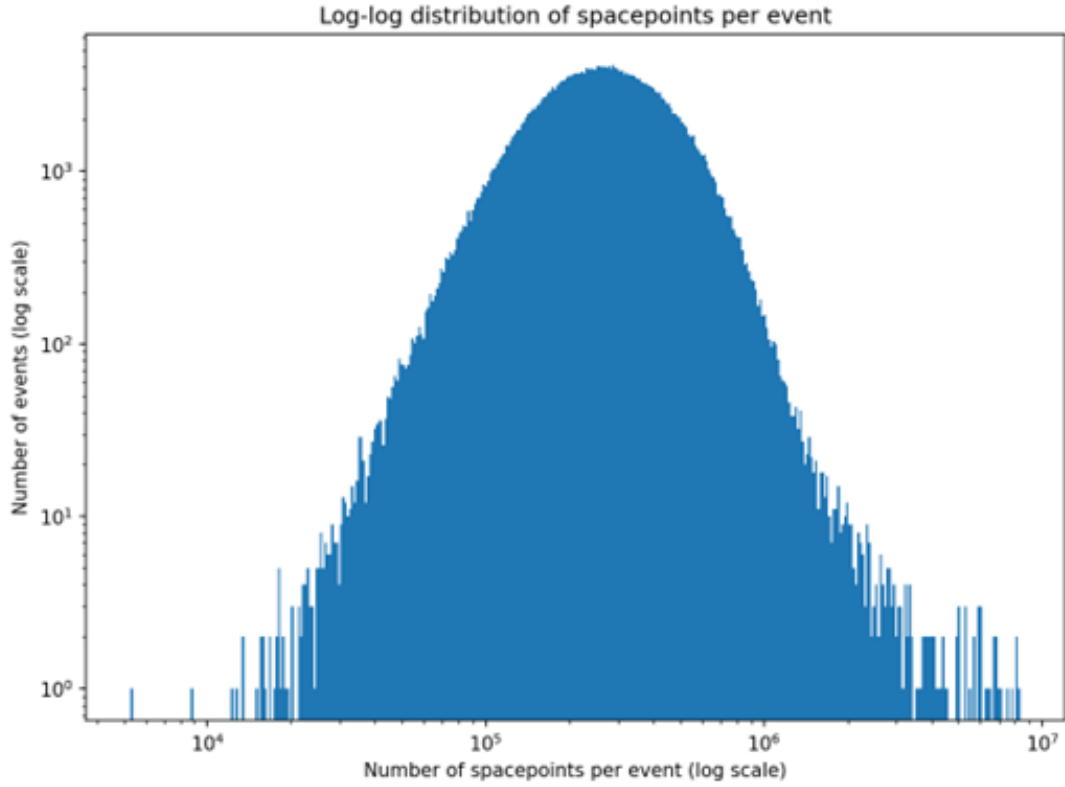


FIG. 6: Log-log distribution of the number of reconstructed spacepoints per event in the real cosmic dataset.

The selected outlier cutoffs were used to remove likely reconstruction failure cases before pretraining.

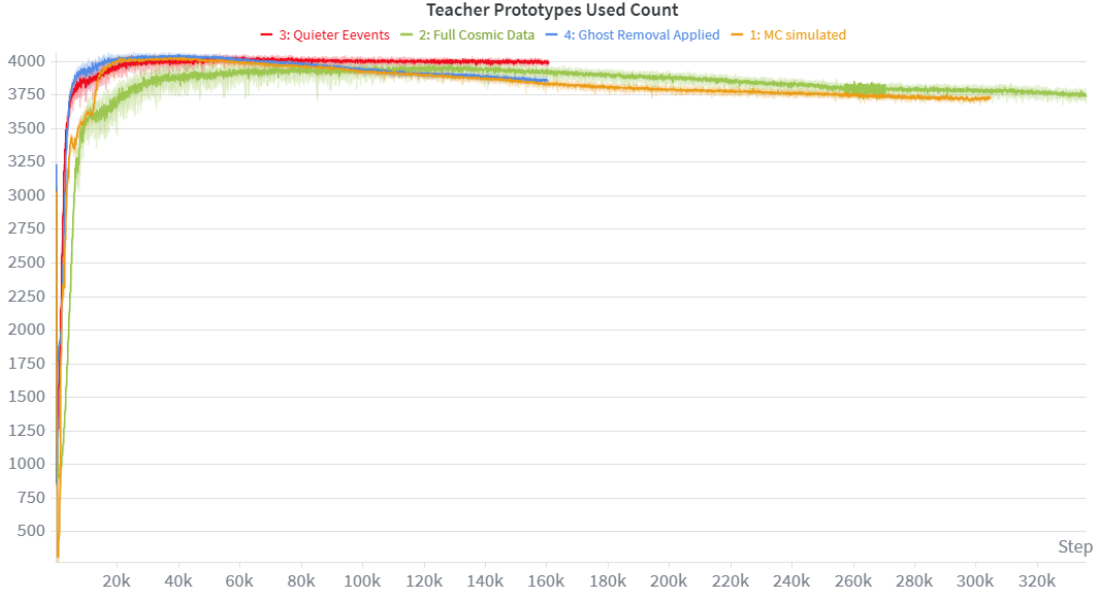


FIG. 7: Teacher prototype usage during Sonata pretraining, monitored as a collapse diagnostic to check that the model used a broad range of learned prototypes rather than collapsing to a small subset. This shows that all four models students used a similar number of formed clusters after a given number of iterations. An iteration is counted each time the model parameters update.

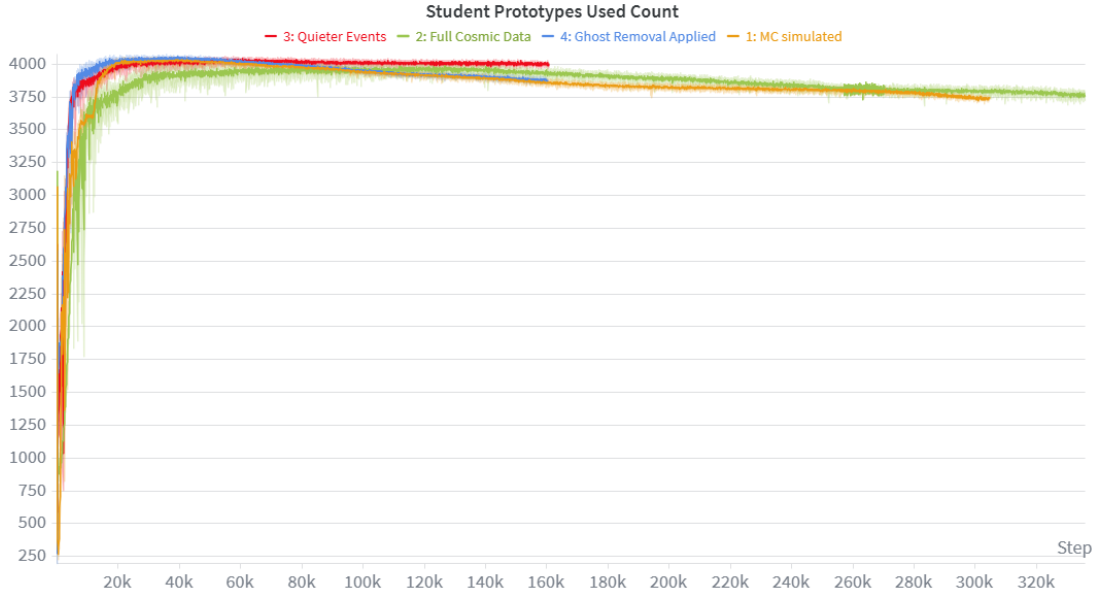


FIG. 8: Student prototype usage during Sonata pretraining. Just as for the teacher the four models students used similar numbers of prototypes at a given number of iterations.